CS 558: Computer Systems Lab
(January-May 2026)

Assignment – 5

Submission deadline: 11:55 PM, Wednesday, 9th April, 2026

Assignment Instructions

- All groups are required to implement every question in the assignment.
- Programs must be developed using **C or C++**.
- In case any problem statement is unclear or incomplete, explicitly state your assumptions along with your solution.
- Submit all source code files bundled into a single compressed file. The file should be named according to your group number (e.g., `Group_4.zip`, `Group_4.rar`, or `Group_4.tar.gz`).
- Only one member per group should submit the assignment via the provided form: https://forms.gle/sJqTsRShxCNyGPFn7
- Evaluation will be conducted offline or through a **viva voce** during your lab session. Be prepared to clearly explain your implementation and approach to the evaluator.
- A plagiarism detection system will be used. Any instance of unfair means will result in **negative marking**, equivalent to the total marks allotted for the assignment.

Question 1: Thread Synchronization – Restaurant Scenario

Consider a popular restaurant in a town that has a seating capacity of **N diners at a time**. However, the number of people waiting outside is significantly larger than N.

The restaurant operates in repeated service cycles as follows:

1. When ready to serve, the restaurant **opens its front door** to allow diners to enter.
2. Diners enter until the restaurant reaches its full capacity (**N diners**).
3. Once full, the restaurant **closes the front door** and begins serving the diners.
4. After the service is completed, the restaurant **opens the back door** to allow diners to exit.
5. Once all diners have exited, the restaurant prepares for the next cycle and repeats the process.

Note: The same diner may participate in multiple service cycles.

Task

Write a **C/C++ multithreaded program** to simulate this scenario, where:

- The **restaurant** is represented by one thread.
- The **diners** are represented by multiple threads.

Synchronization Constraints

Your program must ensure the following:

- A diner can enter **only when the front door is open**.
- The restaurant can begin service **only after exactly N diners have entered**.
- Diners can exit **only when the back door is open**.
- The restaurant can proceed to the next batch **only after all diners from the current batch have exited**.

Question 2: Deadlock-Free Bridge Crossing (Thread Synchronization)

The **Saraighat Bridge** is the only connection between North and South Guwahati and is one of the busiest bridges in the region. A potential deadlock situation can arise if people traveling in **both directions (northbound and southbound)** enter the bridge at the same time. Assume that once a person enters the bridge, they cannot turn back.

Task

Write a **C/C++ multithreaded program** using **semaphores and/or mutex locks** to simulate this scenario and ensure that **deadlock is avoided**.

- Represent:
  - **Northbound people** as one set of threads
  - **Southbound people** as another set of threads
- Once a thread (person) enters the bridge, it should:
  - Sleep for a **random duration** to simulate crossing the bridge
- Your program should support creating **multiple threads** for both directions.

Cases to Handle

Case 1: Directional Exclusivity (One Direction at a Time)

- People traveling in one direction (e.g., northbound) **block** those traveling in the opposite direction (southbound), and vice versa.
- At any given time, the bridge allows traffic **only in one direction**.

Case 2: Non-Blocking Scenario (Concurrent Access Allowed)

- People traveling in one direction do **not prevent** those traveling in the opposite direction from using the bridge.
- Both northbound and southbound people may be on the bridge **simultaneously**, but your design must still **avoid deadlock**.

Objective

Design synchronization logic such that:

- No deadlock occurs in either case
- Threads coordinate safely while accessing the shared resource (the bridge)
- The simulation reflects realistic crossing behavior using delays

Question 3: Process Scheduling in a Distributed E-Commerce System

You are a software developer working on a large-scale **e-commerce platform** that processes millions of transactions daily. The platform operates on a **distributed system** with multiple servers, each responsible for handling different types of services.

System Overview

- The system consists of **n services**, where each service handles a specific type of transaction (e.g., payment processing, order processing, etc.).
- Each service maintains a pool of **m worker threads**.
- Every worker thread has:
  - A **priority level** (used for scheduling decisions)
  - A fixed number of **resources** (maximum number of requests it can process concurrently)
- Incoming requests:
  - Are first assigned to the appropriate **service** based on their **transaction type**
  - Within a service, requests are assigned to worker threads based on **priority and resource availability**

Task

Design and implement a **process scheduling algorithm in C/C++** that efficiently assigns incoming requests to worker threads while optimizing resource utilization.

Scheduling Requirements

Your algorithm must consider:

- **Thread Priority:**
  Higher-priority worker threads should be preferred for processing requests. If unavailable, lower-priority threads may be used.
- **Resource Constraints:**
  Each worker thread can only handle a limited number of requests simultaneously based on its assigned resources.
- **Transaction Type:**
  Different types of transactions may require varying amounts of resources (e.g., payment processing may require more resources than order processing).

Input Format (from Standard Input)

1. Number of services: `n`
2. Number of worker threads per service: `m`
3. Details of each worker thread (one per line):

   ```
   priority_level resources
   ```

4. Details of each request (one per line):

   ```
   transaction_type resources_required
   ```

Output Requirements (to Standard Output)

Your program must display:

- The **order in which requests are processed**
- The **average waiting time** of all requests
- The **average turnaround time** of all requests
- The **number of requests rejected** due to insufficient resources

Additional Metrics (During High Traffic Conditions)

Also report:

- Number of requests that were **delayed (forced to wait)** due to lack of available resources
- Number of requests that were **blocked** due to no available worker threads

Objective

Develop an efficient scheduling strategy that:

- Prioritizes high-priority threads
- Optimally utilizes available resources
- Minimizes waiting and turnaround time
- Handles overload conditions gracefully